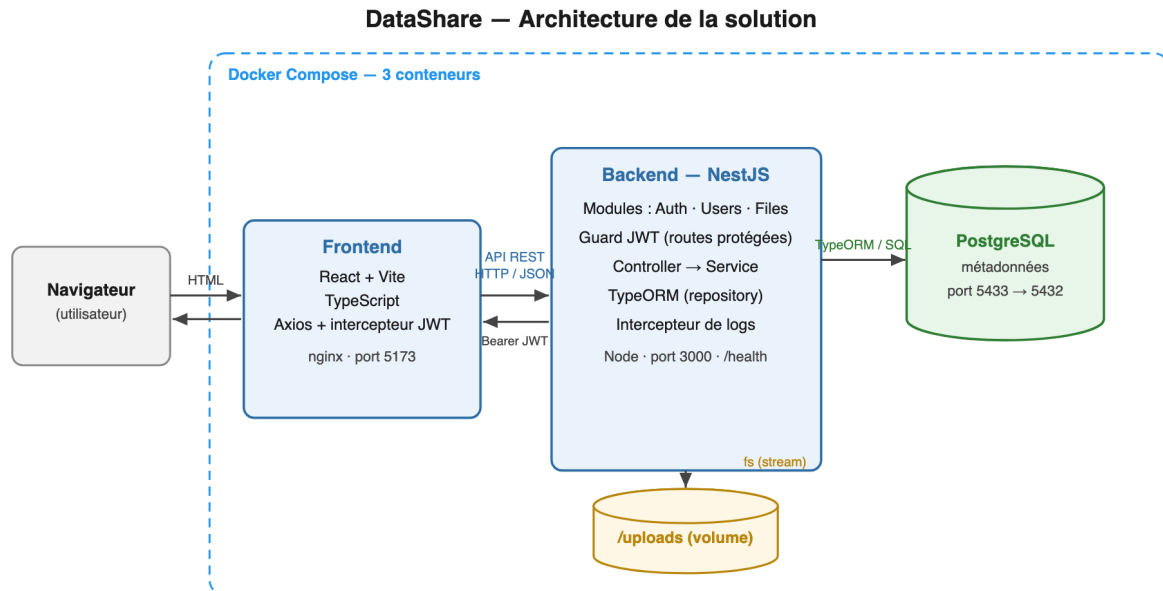


Documentation technique — DataShare

Application web de partage de fichiers par lien temporaire : dépôt d'un fichier, génération d'un lien unique (protégeable par mot de passe) qui expire au bout de 1 à 7 jours.

1. Architecture de l'application

Architecture 3 couches, entièrement conteneurisée : Frontend React+Vite (nginx) → Backend NestJS → PostgreSQL. Le front parle au back via une API REST (HTTP/JSON) avec un JWT en en-tête ; le back lit/écrit les métadonnées via TypeORM et stocke les fichiers sur disque en streaming.



2. Justification des choix technologiques

Backend — NestJS (TypeScript)

Architecture modulaire imposée, TypeScript natif, injection de dépendances, validation, tâches planifiées et tests intégrés. Alternatives écartées : Spring (plus lourd), Symfony, Express (trop minimal).

Langage — TypeScript

Le typage détecte les erreurs avant l'exécution ; code plus clair et maintenable qu'en JavaScript.

Frontend — React + Vite

React est le standard du marché (large écosystème) ; Vite offre un démarrage et un build très rapides. Alternatives : Angular, Vue.

Base de données — PostgreSQL

Relationnelle, adaptée à « un utilisateur possède plusieurs fichiers » ; garantit relations et unicité (email). Alternative : MongoDB, écartée (modèle relationnel).

ORM — TypeORM

Intégration native NestJS ; entités annotées = tables. synchronize en dev, migrations en prod.

Auth — JWT + bcrypt

JWT stateless (pas de session serveur, scalable) ; bcrypt hache les mots de passe (lent par conception, résistant à la force brute).

Stockage fichiers

Fichiers sur disque, métadonnées en base : chacun fait ce qu'il fait le mieux. Réception via Multer (limite 1 Go).

Conteneurisation — Docker Compose

Environnement identique partout, déploiement reproductible (3 services).

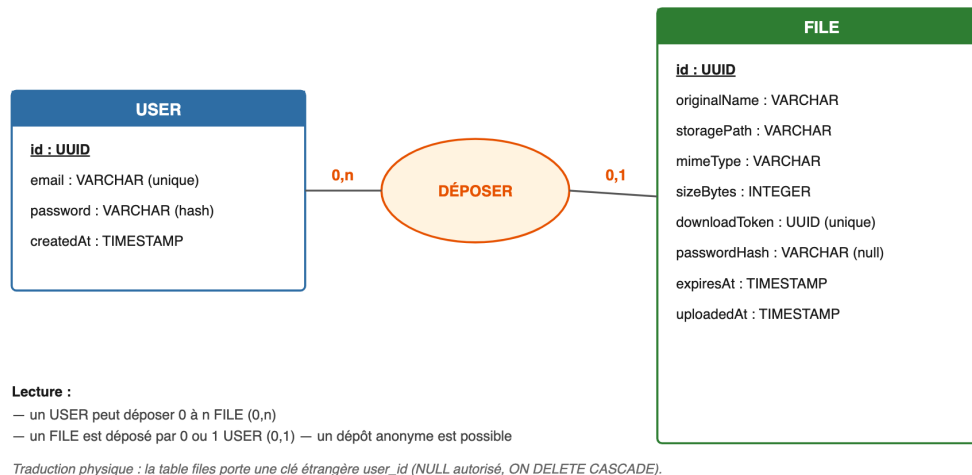
Tests — Jest, Supertest, Cypress, k6

Pyramide des tests (unitaire / intégration / e2e) + test de charge.

3. Modèle de données

Deux entités : USER (1) — (n) FILE, reliées par une clé étrangère user_id (nullable, ON DELETE CASCADE).

DataShare — Modèle Conceptuel de Données (MCD)



4. Documentation des endpoints (API)

Contrat d'interface complet au format OpenAPI 3.0.3 (openapi.yaml ; doc interactive docs/api.html).

Méthode	Route	Rôle	Auth
GET	/health	Health check	—
POST	/auth/register	Créer un compte	—
POST	/auth/login	Se connecter (JWT)	—
POST	/files/upload	Téléverser un fichier	JWT
GET	/files	Lister mes fichiers	JWT
DELETE	/files/:id	Supprimer un fichier	JWT
GET	/d/:token	Infos d'un lien	—
POST	/d/:token/download	Télécharger	—

5. Sécurité et gestion des accès

- Mots de passe hachés (bcrypt), jamais en clair ; JWT signé, secret en variable d'environnement.
- Liens = UUID v4 non devinables + expiration ; refus des exécutables (magic number).
- Cloisonnement par propriétaire ; limite de taille (anti-DoS) ; gestion des erreurs (404/403/401). Détail : SECURITY.md.

6. Qualité, tests et maintenance

- 57 tests (47 unitaires + 7 intégration + 3 e2e), couverture ~99 %.
- Test de charge k6 (médiane ~100 ms), logs structurés. Détail : TESTING.md, PERF.md.
- Plan de maintenance (mises à jour, audit, sauvegardes) : MAINTENANCE.md.

7. Installation et exécution

Prérequis : Node 20+, Docker. Démarrage :

```
./setup.sh  
cd backend && npm run start:dev  
cd frontend && npm run dev
```

Ou tout en conteneurs : docker-compose up -d --build. Détail : README.md.

8. Utilisation de l'IA dans le développement

Tâches confiées à l'IA, rôle de supervision et correctifs apportés : voir AI_USAGE.md. Posture : l'IA assiste, l'étudiante définit, relit, teste et valide.